MOGA+LSA: How the Pipeline Works

A plain-language walkthrough of the cluster-head selection and routing pipeline, and the GraphSAGE variant we compared it against.

1. The problem we are solving

A wireless sensor network is a field of battery-powered sensor nodes that need to get their readings back to a base station (also called the **sink**). Radio transmission is what drains the batteries, and the cost grows very fast with distance, so having all 100 nodes shout directly at the base station would kill the network quickly.

The fix is **clustering**. A small number of nodes are promoted to **cluster heads** (CHs) each round. Ordinary nodes send their reading a short distance to their nearest cluster head, the cluster head merges everything it received into one message, and only the cluster heads pay the expensive long-distance transmission to the sink.

That leaves two questions to answer every round, and the two halves of this pipeline answer one each:

- **Which nodes should be cluster heads?** — answered by MOGA, a genetic algorithm.
- **How should those cluster heads get their data to the sink?** — answered by LSA, a separate search algorithm.

2. The pipeline at a glance

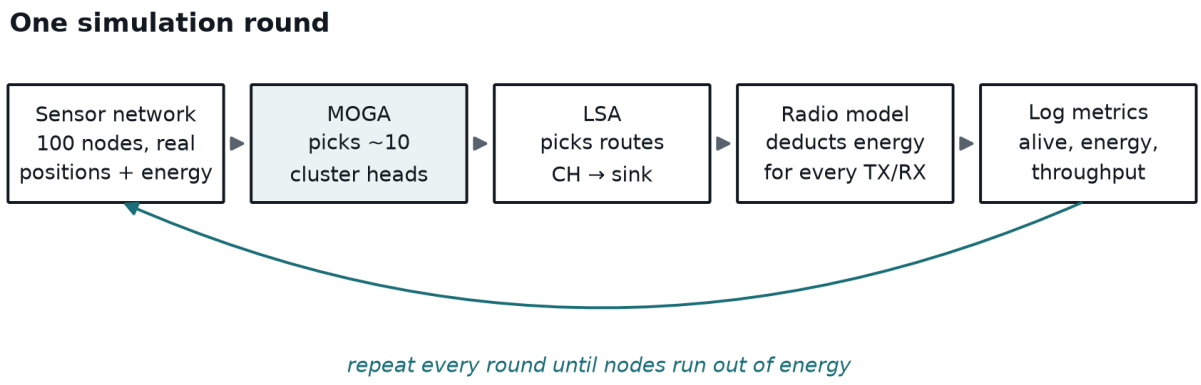


Figure 1. One simulation round. Everything inside repeats thousands of times until the nodes run out of energy.

The important thing to notice is that this whole sequence runs **every single round**. Cluster heads are not chosen once at the start and kept forever — they are re-chosen from scratch each round, because batteries drain and a node that was a great cluster head earlier may be nearly flat now.

3. Step one: load the network

We take 100 real sensor records from the Kaggle WSN dataset. Each node keeps its real X/Y position on a 100 m × 100 m field, its real starting battery level (rescaled so the average node starts at 0.5 joules), and eleven real sensor attributes such as signal strength, noise level, temperature and packet loss rate.

The sink is placed at the centre of the sampled nodes. Each random seed draws a different 100-node sample from the dataset, which is how we get variation between runs.

4. Step two: MOGA picks the cluster heads

MOGA stands for **Multi-Objective Genetic Algorithm**. A genetic algorithm is a trial-and-error search loosely modelled on evolution: try a batch of candidate answers, keep the good ones, combine them to make new ones, add a little randomness, and repeat until the batch has improved.

The vocabulary

- **Chromosome** — one candidate answer. Here, one chromosome is one proposed set of about 10 cluster heads.
- **Population** — the batch of chromosomes being tried at once. We use 20.
- **Generation** — one pass of score / keep the best / breed / mutate. We run 10 generations per round.
- **Fitness** — a single number scoring how good a chromosome is. In our setup, **lower is better**.

A common point of confusion: the 20 and the 10 are different counts stacked on top of each other. There are **20 competing proposals**, and **each one of those proposals contains its own 10 cluster heads**. The very first 20 proposals are built by picking 10 node IDs completely at random (no duplicates within one proposal). There is no cleverness at that starting point — the intelligence only comes from what happens next.

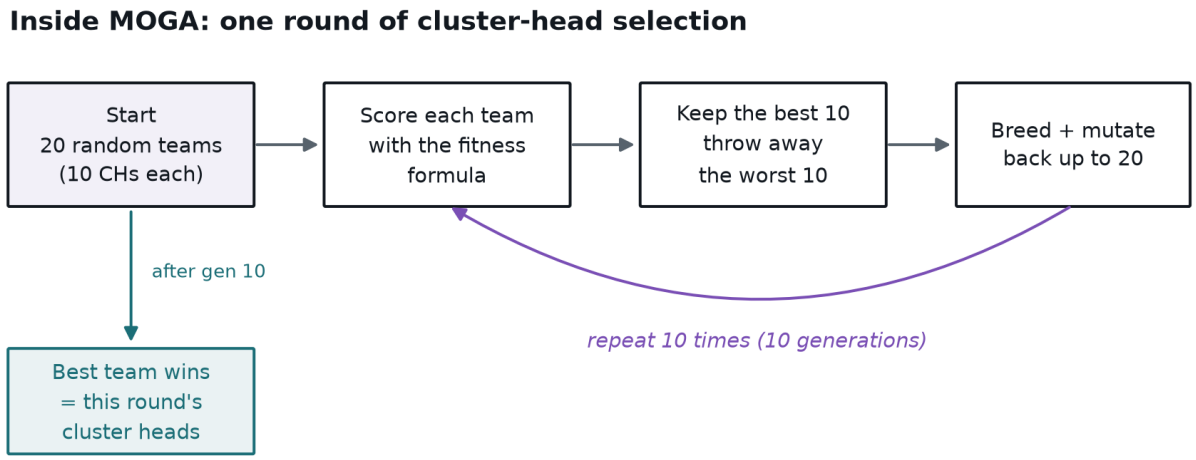


Figure 2. What MOGA does inside a single round.

The fitness formula, in words

Each proposal is scored on three things added together:

- **Member-to-CH distance** — how far each ordinary node is from its nearest cluster head, summed up. Smaller is better.
- **CH-to-sink distance** — how far the cluster heads are from the base station. Smaller is better.
- **Residual energy of the cluster heads** — how much battery the chosen cluster heads still have. This one is **subtracted**, because more battery is better and we are minimising the total.

All three are normalised onto a 0–1 scale before being added, so that no single term dominates the score just because it happens to be measured in bigger units. In plain terms the formula is asking: *pick cluster heads that are close to their members, close to the base station, and still have plenty of battery.*

How breeding (crossover) works

Take two surviving proposals and cut both at the same random point, then glue the front of one to the back of the other. With shortened 6-head lists for readability:

Parent A	`12, 45, 7`	`|`	`88, 3, 61`
Parent B	`9, 22, 50`	`|`	`4, 77, 15`
Child	`12, 45, 7`	`+`	`4, 77, 15`

The idea is that two proposals which both scored reasonably well probably each contain some genuinely good cluster-head picks mixed with some mediocre ones. Splicing them gives the child a chance of inheriting the good parts of both. Two housekeeping rules apply: if the splice produces a duplicate node, the duplicate is dropped, and if that leaves fewer than 10 cluster heads, random nodes are added to pad it back up.

Mutation then gives each child a 20% chance of having one of its cluster heads swapped for a different random node. This keeps a bit of fresh randomness in the population so the search does not get stuck refining the same narrow set of ideas.

After 10 generations of this, the single best-scoring proposal wins and its 10 nodes become the actual cluster heads for the round. The other 19 are discarded — they existed only to help the search.

5. Step three: LSA picks the routes

LSA stands for **Lightning Search Algorithm**. It belongs to the same family as the genetic algorithm — start with random candidate answers, improve them over repeated iterations — but it is modelled on how a lightning strike probes for the easiest path to the ground rather than on evolution.

Its job here is different from MOGA's. MOGA has already decided *who* the cluster heads are. LSA decides *how their data gets home*. A cluster head does not have to transmit straight to the sink; it may be cheaper to hand the data to another cluster head that is closer to the sink and let that one forward it. This is the same logic as passing a message to a friend nearer the destination instead of shouting all the way there yourself, because shouting a long distance costs disproportionately more battery.

LSA searches over 30 iterations with 15 candidate routing plans, balancing three things at once: how much data actually arrives, how much battery the plan costs, and how long it takes.

6. Step four: energy is deducted

Now the round actually happens and everyone pays for it. The cost is not a flat fee — it depends on how many bits were sent and how far they travelled:

- **Sending** costs a fixed electronics charge per bit (50 nJ/bit), plus an amplifier charge that grows with distance^2 for short hops and distance^4 for long hops (the cutoff is about 87.7 m).
- **Receiving** costs only the electronics charge (50 nJ/bit). Distance does not matter when receiving.
- **Aggregating** costs a cluster head an extra 5 nJ/bit to merge everything its members sent into one outgoing message.

So in one round an ordinary member pays once to transmit. A cluster head pays to receive every member's message, pays to aggregate them, then pays to transmit the result onward. A relaying cluster head pays to receive and then to transmit again. Any node whose battery reaches zero is marked dead and takes no further part in the simulation.

This distance-squared and distance-to-the-fourth behaviour is exactly why cluster-head placement matters so much, and it is worth keeping in mind for section 8.

7. Step five: repeat and record

Steps two to four repeat every round, up to 13,000 rounds in our longest run. After each round we record how many nodes are still alive, the average battery remaining, the delivery rate and the average delay. Plotting the alive-node count over time is what gives us the three standard lifetime milestones:

- **FND** (First Node Death) — the round the first node dies.
- **HND** (Half Node Death) — the round half the network is dead.
- **LND** (Last Node Death) — the round the final node dies.

8. The GraphSAGE variant

Everything above describes the stock protocol. The variant we compared it against changes exactly one thing.

GraphSAGE is a neural network designed to work on data that is connected in a network rather than laid out in a flat table. It produces an **embedding** for each node: a list of 16 numbers that acts like a fingerprint, summarising not just the node itself but also what its neighbours look like.

How it is trained

- First we build a graph by connecting each node to its 8 physically nearest neighbours on the map. This graph only decides who counts as whose neighbour; separately, each node carries 14 numbers (position, battery and the eleven sensor readings) that the network reads.
- Then we train the network on a puzzle: *given two nodes' fingerprints, can you tell whether they are actually neighbours?* Real neighbours are the positive examples; random pairs are the negative examples.
- Nobody ever tells the network what a “correct” fingerprint looks like, so this is called **unsupervised** training. Getting good at the puzzle forces similar nodes to end up with similar fingerprints, which is the useful part.
- Training runs for 100 passes over the data, once at the very start of the simulation. The model is then frozen and the same fingerprints are reused for all 13,000 rounds.

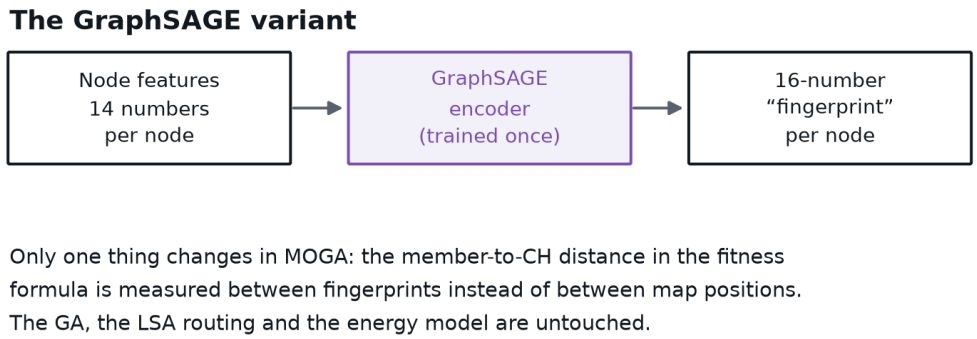


Figure 3. The single point at which GraphSAGE enters the pipeline.

The only change in the pipeline is in the first term of the fitness formula. Instead of measuring member-to-cluster-head distance on the map, we measure it between the 16-number fingerprints. The genetic algorithm, the LSA routing, the other two fitness terms and the energy model are all completely unchanged, so any difference in the results comes from this substitution alone.

9. What we found

The two methods performed essentially the same. Over 3,000 rounds no node died in either version, and the difference in remaining battery was 0.077% — smaller than the variation between random seeds, and not statistically significant ($p = 0.123$).

Over the extended 13,000-round run the two lifetime milestones moved in opposite directions: FND got 3.7% worse with GraphSAGE while HND got 0.9% better. LND was never reached by either version.

Metric	MOGA+LSA	MOGA+LSA + GraphSAGE
Residual energy at 3,000 rounds	0.33127 J	0.33102 J (p = 0.123)
FND	6,700 rounds	6,450 rounds (-3.7%)
HND	8,800 rounds	8,875 rounds (+0.9%)
LND	not reached	not reached

Why the null result is interesting rather than disappointing. Recall from section 6 that transmission energy depends on *physical* distance. The member-to-CH distance term is therefore not an arbitrary measure of closeness — it is a direct stand-in for the energy the formula is trying to save. Replacing it with a learned fingerprint distance cuts the fitness formula loose from the energy model, and the extra structural information does not make up for what was lost.

That is a useful finding for the wider study: it suggests graph embeddings help when they are **added alongside** physically meaningful terms, rather than **substituted in place of** them. A second possible explanation is the training signal — our model learns which nodes look alike, but is never shown whether a given cluster-head choice actually shortened the network's life.

10. Where the code lives

`wsn_sim/wsn_dataset.py`	Loads the Kaggle dataset, samples 100 nodes
`wsn_sim/moga.py`	The genetic algorithm, fitness, crossover, mutation
`wsn_sim/lsa.py`	Lightning Search Algorithm routing
`wsn_sim/network.py`	Radio energy constants and network state
`wsn_sim/simulate.py`	The round loop and metric logging
`wsn_sim/graphsage_embed.py`	GraphSAGE model and its training
`wsn_sim/moga_embedding.py`	Wires the embeddings into MOGA
`run_compare.py`	3 seeds x 3,000 rounds comparison
`run_extended_lifetime.py`	1 seed x 13,000 rounds, for FND/HND/LND
`ns3_wsn_sim.py`	Exports the ns-3 / NetAnim packet-level traces

To reproduce the numbers in section 9: run `python run_compare.py` for the 3-seed comparison and `python run_extended_lifetime.py` for the lifetime milestones. Both write their results to JSON files that the plotting scripts read.